

FACULTAD DE INGENIERÍA

UNIVERSIDAD AUTÓNOMA DE CAMPECHE

CONCURRENCIA Y PARALELISMO

REPORTE

MAURICIO CASTILLO OLOARTE

SERGIO A NOH PUCH

ITS 5 A

Cómo lo hice

Hice el programa EscrituraBuffered.java con los ocho flujos. Cada método mide su propio tiempo con System.nanoTime() desde que abre el flujo hasta que lo cierra, y lo imprime en ms. El try-with-resources ya hace el flush al cerrar. El valor de n se puede cambiar en main o pasarlo como argumento al ejecutar, por ejemplo java EscrituraBuffered 1000000.

- Texto: se escriben n líneas con el número i y un salto de línea. La lectura es carácter por carácter con read, tanto en FileReader como en BufferedReader, para que la comparación sea justa.
- Binario: se escriben n bytes con write y se leen byte por byte con read.
- Corrí el programa 3 veces por cada valor de n: 100,000, 1,000,000 y 10,000,000. No descarté la primera ejecución porque cada vez arrancaba una JVM nueva.

Resultados

n = 100,000, tiempos en ms

Flujo utilizado	Ejec. 1	Ejec. 2	Ejec. 3	Promedio
FileWriter	66.65	81.97	61.42	70.01
BufferedWriter	24.72	35.69	31.17	30.53
FileReader	71.27	74.92	88.24	78.14
BufferedReader	26.47	25.16	26.03	25.89
FileOutputStream	47.33	47.12	53.71	49.39
BufferedOutputStream	14.77	7.43	7.93	10.04
FileInputStream	40.23	39.59	37.44	39.09
BufferedInputStream	8.92	6.48	7.79	7.73

n = 1,000,000, tiempos en ms

Flujo utilizado	Ejec. 1	Ejec. 2	Ejec. 3	Promedio
FileWriter	156.15	205.53	157.76	173.15
BufferedWriter	96.41	95.11	118.20	103.24
FileReader	354.57	342.78	351.60	349.65
BufferedReader	152.03	192.34	159.86	168.08
FileOutputStream	479.20	436.14	442.95	452.76
BufferedOutputStream	35.70	38.61	30.16	34.82
FileInputStream	390.77	349.28	344.53	361.53
BufferedInputStream	34.20	29.34	32.13	31.89

n = 10,000,000, tiempos en ms

Flujo utilizado	Ejec. 1	Ejec. 2	Ejec. 3	Promedio
FileWriter	827.02	821.89	833.94	827.62
BufferedWriter	497.30	471.43	535.01	501.25
FileReader	3456.30	3505.18	3765.30	3575.59
BufferedReader	1736.14	1717.50	1719.44	1724.36
FileOutputStream	4703.60	4636.17	4832.74	4724.17
BufferedOutputStream	236.47	235.85	256.14	242.82
FileInputStream	3642.55	3661.55	3601.05	3635.05
BufferedInputStream	221.18	223.74	223.24	222.72

Con n = 10,000,000 el archivo de texto pesa unos 79 MB y el binario 10 MB.

Análisis

1. Diferencia entre flujos con buffer y sin buffer, y por qué.

Los flujos con buffer fueron más rápidos en todos los casos. En binario la diferencia es enorme: con 10 millones de bytes, FileOutputStream tardó casi 4.7 s y BufferedOutputStream unos 0.24 s, o sea unas 19 veces menos. En lectura, FileInputStream tardó 3.6 s contra 0.22 s de BufferedInputStream, unas 16 veces menos. Pasa porque sin buffer cada write o read es una llamada al sistema operativo para un solo byte, y esa llamada es muy cara. Con buffer los bytes se juntan en memoria y se mandan al disco en bloques, así que se hacen muchísimas menos llamadas. En texto la mejora fue menor. BufferedWriter fue más o menos 1.6 veces más rápido que FileWriter y BufferedReader unas 2 veces más rápido que FileReader. Creo que es porque FileWriter y FileReader ya usan internamente un codificador con su propio buffer de 8 KB, así que no hacen una llamada al sistema por cada carácter, aunque el buffered igual ayuda.

2. Cómo cambia el tiempo al aumentar n, ¿es lineal?

Más o menos sí, aunque no exacto. Al pasar de 1 millón a 10 millones, que son 10 veces más datos, FileOutputStream pasó de 453 ms a 4724 ms, unas 10.4 veces más, y FileInputStream de 362 ms a 3635 ms, unas 10 veces más. Eso es casi lineal. BufferedOutputStream pasó de 35 ms a 243 ms, unas 7 veces, menos que lineal. Probablemente con n chico pesa más el costo fijo de abrir el archivo y de que la JVM todavía no ha optimizado el código. Entre 100,000 y 1,000,000 también se ve que los tiempos chicos no escalan igual por ese costo fijo. En general lectura y escritura crecieron parecido, pero en texto la lectura fue bastante más lenta que la escritura. Con 10 millones, FileReader tardó 3.5 s contra 0.8 s de FileWriter. Leer carácter por carácter con read obliga a decodificar y devolver un solo carácter en cada llamada, mientras que en la escritura se mandan cadenas completas de una vez con write y un String completo.

3. ¿Hay diferencia entre caracteres y bytes?

Sí, aunque hay que tener cuidado al comparar porque en mi prueba no se escribió lo mismo. En texto cada elemento es una línea con varios caracteres y en binario es un solo byte. Aun así se ve que sin buffer el binario es más lento por elemento, porque cada byte es una llamada al sistema, pero con

buffer el binario es el más rápido de todos ya que no hay que codificar nada. En texto hay un costo extra, que es convertir caracteres a bytes según el charset por defecto al escribir y al revés al leer. Además el archivo de texto es mucho más grande, 79 MB contra 10 MB.

4. Tamaño del buffer por defecto y si se puede cambiar.

En BufferedOutputStream, BufferedInputStream, BufferedWriter y BufferedReader el buffer por defecto es de 8192, o sea 8 KB. Son bytes en los de bytes y caracteres en los de texto. Sí se puede cambiar porque todos tienen un constructor que recibe el tamaño, por ejemplo new BufferedOutputStream(fos, 65536) o new BufferedReader(fr, 16384).

5. ¿Por qué BufferedReader tiene readLine y BufferedInputStream no?

Porque readLine trabaja con la idea de línea, que es un concepto de texto, caracteres terminados en salto de línea. BufferedReader lee caracteres, así que puede reconocer el salto de línea y devolver un String con toda la línea. BufferedInputStream solo maneja bytes sin ningún significado, y en un archivo binario un byte con valor 10 no necesariamente es un salto de línea. La implicación al leer texto es que conviene usar BufferedReader y readLine, porque es más cómodo y rápido que armar las líneas a mano byte por byte, y además se encarga de la decodificación de caracteres.

Conclusión

Usar buffer vale la pena casi siempre. Con muy poco código extra, que es envolver el flujo en un Buffered, se reducen mucho los accesos al disco, y en mis pruebas con bytes el tiempo bajó más de 15 veces. En texto la mejora fue menor porque FileWriter y FileReader ya traen algo de buffer interno, pero aun así fue de 1.6 a 2 veces más rápido, y BufferedReader además da readLine. Lo único a cuidar es cerrar bien el flujo con try-with-resources para que el buffer se vacíe y no se pierdan datos. Una limitación de mis resultados es que son mediciones simples con nanoTime. La JVM optimiza el código mientras corre y el disco de esta máquina puede tener su propio caché, así que los números exactos pueden variar en otra computadora, aunque la tendencia debería ser la misma.